Practical-01

Aim: Creating Environment Setup for React

Theory:

React is a powerful JavaScript library used for building user interfaces. Setting up a development environment is essential to get started with building React applications. This process involves installing key software such as Node.js (JavaScript runtime), npm (Node Package Manager), and using Create React App to scaffold a React project quickly.

Tools and Software:

- **Node.js:** A JavaScript runtime that allows us to run JavaScript outside the browser. It includes npm, the Node Package Manager.
- **npm:** A package manager for JavaScript that helps install libraries and manage dependencies.
- **Create React App:** A tool that sets up a new React project with a preconfigured build environment.
- **Code Editor (Optional):** A text editor such as Visual Studio Code (VS Code) for writing and editing code.

Procedure:

1. Install Node.js and npm

Node.js and npm are required to run React and manage the packages and dependencies of a React project.

- **Step 1:** Go to the official Node.js website: https://nodejs.org/.
- **Step 2:** Download the latest LTS version (Long-Term Support).
- **Step 3:** Run the installer and follow the on-screen instructions to install Node.js and npm.
- **Step 4:** After installation, verify the installation by opening your terminal/command prompt and running the following commands:
- node -v
- npm -v

You should see version numbers for both Node.js and npm.

2. Install Create React App

Create React App is a command-line utility that sets up a new React project with a default configuration.

- **Step 1:** Open the terminal or command prompt.
- **Step 2:** Install Create React App globally by running the following command:
- npm install -g create-react-app

This command installs Create React App so it can be used anywhere on your system.

3. Create a New React Project

Once Create React App is installed, you can generate a new React project.

- **Step 1:** In the terminal/command prompt, navigate to the folder where you want your React project to be created.
- **Step 2:** Create a new React app by running:
- create-react-app my-react-app

Replace my-react-app with the name of your project. This will create a folder with that name, containing all the necessary files to get started with React.

- **Step 3:** Navigate into the newly created project folder:
- cd my-react-app

4. Start the Development Server

To view your app in the browser and begin development, you need to start the development server.

- **Step 1:** In the terminal, run the following command:
- npm start

This will start the development server and open the React application in the default web browser at http://localhost:3000/.

5. Open the Project in a Code Editor

To start editing the code, open the newly created project in a code editor.

- **Step 1:** Install Visual Studio Code (or any code editor of your choice) from https://code.visualstudio.com/.
- **Step 2:** Open the my-react-app folder in VS Code by either launching VS Code and opening the folde

6. Modify the Project

You can now start editing the code in the src/App.js file, which contains the main component for your app. Add new components, update the existing code, and customize your app.

7. Install Additional Packages (Optional)

React projects often require external libraries for various features. For example, to add React Router for routing, you can install it via npm.

- **Step 1:** Run the following command to install React Router:
- npm install react-router-dom

8. Build the Project for Production

Once development is complete, you can build your app for production to deploy it to a server.

- **Step 1:** Run the following command to generate a production-ready build:
- npm run build

This will create an optimized version of your app inside the build/ directory, ready for deployment.

Conclusion:

By completing this experiment, you have successfully set up a development environment for React. You are now ready to begin building more advanced React applications. With Create React App, you can quickly set up and start developing without needing to worry about manual configuration of build tools such as Webpack or Babel. This setup will provide a solid foundation for any React-based project.

Practical – 03

Aim:

Write a program to create a simple calculator application using React JS.

Theory:

A calculator application is a basic React program that performs arithmetic operations such as addition, subtraction, multiplication, and division.
This program demonstrates how React components can interact with HTML elements and handle button click events. The calculator takes input from the user and displays the result dynamically using JavaScript functions without using state or hooks.

Procedure:

Create a React Application

Open the terminal or command prompt.
Run:

npx create-react-app calculator-app

Navigate into the project folder using:

cd calculator-app

Start the React Application

Run:

npm start

The application opens in the browser at:

http://localhost:3000

Modify App.js File

Conclusion:

By completing this experiment, we learned how to create a simple calculator application using React JS without using state or hooks. This program demonstrates basic event handling and interaction with HTML elements in React. It helps in understanding the fundamentals of React components and JavaScript logic.

Conclusion:

This practical helped in understanding the core components of React such as components, state, props & event-handling. It also demonstrated how can React JS be used for single pages in Real World.

Page No.

Date / /20

Procedure:

step 1) Set Up React Enviroment
- Install Node JS and npm
- Create App using
npm create vite@latest resume

Step 2) Create Components
Design for
- Name, Email, Phone
- Educational details
- Work Experience, Skills

step 3) Pass data using props

step 4) Create Resume preview and verify formatting ~~using~~ ~~css~~

step 5) Add Styling
- Use css for styling
- Make Resume look professional

Page No.

Date / /20

Practical 2

Aim: To design & develope a Resume Webpage using React

Theory:

React is a popular JS library developed by Facebook for building UI, single page applications.

In this project, React is used to create a Resume where:

- Components are used to divide the UI into smaller reusable parts such as Header, Education, Experience, skill, etc.
- State Management is used to store user input dynamically
- Props are used to pass data between components.
- Event Handling is used to capture user input
- Conditional rendering helps in displaying Resume sections based on user inputs.

Page No.

Date / /20

Feature	Function Component	Class Component
Syntax	Simple Function	ES6 class
State	UseState hook	this.state
life Cycle	UseEffect	lifecycle method
Code length	short	longer

Conclusion :

Thus, we studied & implemented function & class component in React. Function components are simpler & widely used, while class components provide lifecycle methods. Both help in building reusable & modular UI

Page No.

Date / /20

start the React Application

Run:

npm start

The application open in the browser at:
http://localhost:3000

Modify App.js file.

Conclusion:-

By completing this experiment, we learned how to create a simple login and registeration page using React JS without using state or hooks. This programs demonstrates basic React component structure, event handling, and form input using JavaScript. It helps in understanding the fundamentals of building interactive user interface using React.

28

RAJ

Page No.

Date / /20

Practical 7

State and Lifecycle Management in React

Theory:

React is a popular JavaScript library used to build user interface. Two important concepts in React are state and lifecycle, which help manage dynamic data and control how components behave during their existence.

What is State in React?

State is a built in feature of react that allows components to store and manage data that can change over time. When state of componet changes, React automatically updates the UI to reflect those changes.

Key characteristics

- State is managed within a component
- It is mutable (can be changed)
- Changes in state trigger re-rendering.
- Used for dynamic and interactive data

Ex. Counter values, form inputs, etc.

Page No.

Date / /20

Types of States

1) Local State
   - Used within single component
   - Not shared with other components

2) Global state
   - Shared across multiple components
   - Managed using tools like Context API

What is "Lifecycle" in React

The lifecycle of React component refers to series of stages a component goes through from creation to removal from the screen (DOM).

Phases : 1) Mounting phase
   - Creation of component

2) Updating phase
   - Change in state

3) Unmounting phase
   - Component is removed from DOM.

Best Practices
- Keeps state minimal & relevant
- Avoid unnecessary updates
- Properly manage lifecycle behavior
- Clean up resources when component is removed

Page No.

Date / /20

Conclusion:

State and lifecycle management are fundamental concepts in React. State allows component to store & update dynamic data, while lifecycle methods control how components behave at different stages.

Page No.

Date / /20

Conclusion:

This practical demonstrates how React uses lists & keys to handle dynamic data rendering. Understanding this concept is essential for building scalable and efficient user interface in modern web application.

Page No.

Date / /20

Practical 6

Aim: To implement component inheritance in Reat by passing a message from parent component to a child component & displaying it.

Theory:

In React, traditional inheritance is not commonly used. Instead, React uses Component composition and props to achieve inheritance like behavior.

Props are used to pass data from a parent component to a child component. This allows components to communicate & reuse functionality. In this practice, a message is passed from another parent component to child component, demonstrating inheritance-like behavior.

Component Used:

- Parent component : App.jsx
- Child component : Home.jsx
- Nested child component: Product Card.jsx

Page No.

Date / /20

Ex.

Ex A list of students is considered as an example. The data is stored in an array of objects, where each object contains details like ID, name, courses.

The array is processed using the map() function. For each object:

- A list element (<li>) is created.
- Student details are displayed inside it.
- A unique key is assigned using the student's IDs.

This approach avoids repetition & makes UI dynamic.

Working Flow:

1) Create an array of data (student /items)
2) Use map() to iterate over the array.
3) Return JSX element for each item.
4) Assign a unique key to each element
5) Render the list inside & container (like <ul>)

Advantages

- Reduces code repetition
- Improves readability & maintainability
- Enables dynamic UI updates
- Enhances performance

Page No.

Date / /20

Practical 4

* Aim: Create a simple login & Registeration page using React JS.

* Theory:

A login and registeration page is used to collect user credentials such as username, email and password.

In React JS, user interactions like button clicks can be handled using React components and HTML form elements without using state or hooks. User input is accessed directly using DOM methods.

* Procedure:

Create a react application

Open the terminal or command prompt.

Run:

```
npx create-react-app login-app
```

Navigate into project folder using

```
cd login-app
```

RAJ

Page No.

Date / /20

Practical 8

Aim:- To study & understand the concept of lists & keys in React JS & how they are used to render multiple elements dynamically.

Theory:

In react, displaying multiple similar elements (like names, products, or records) is efficiently handled using lists. Instead of manually writing repeated HTML elements, React allows developers to use JS arrays & iterate over them.

The most commonly used method is map(), which transforms each element of an array into a React element.

Keys are special attributes provided to each list elements. They help React identify which items have changed, been added, or removed. This improves performance during re-rendering.

- Lists are created using arrays
- map() is used to iterate over data
- Each element must have a unique key.
- Keys should be stable & unique (like IDs) & not indexes.

Page No.

Date / /20

Practical 5

Function & Class Component

Aim: To create and understand function component & class component in React & implement them in a grocery store application

Theory:

Component: Components are building blocks of a React Application. They help divide UI into reusable parts.

1. **Function Component**

Function components are simple JS functions that return JSX (UI)

- **Features:**
  - Easy to write
  - Use hooks like useState, useEffect.
  - Modern approach

```
function Welcome() {
    return <h1> Welcome to Store </h1>
}
```

Page No.

Date / /20

2. Class Components:

Class components are ES6 classes that extend react component.

- Features:
  - Use this.state for data
  - Use lifecycle method like ComponentDidMount()

Procedure:

1. Create a React Project
2. Create Function Component (Home, Cart, etc)
3. Create Class component.
4. Pass data using props.
5. Render components in App.js

Result: The function and class components were successfully created and implemented in the grocery store app^n. The component were able to render UI and pass data using props.

Page No.

Date / /20

Steps / Procedure :

1. Create react application
2. Define a message in Parent Component
3. Pass the message to the child component using props.
4. Further pass the message to nested child.
5. Display the message in child component.

Result: The message was successfully passed from parent component to the child component & displayed correctly. This demonstrates inheritance-like behavior using props in React.

Conclusion: Thus, component inheritance was implemented in React Using Props. The parent component passed data to the child component & child component rendered the message successfully.